ارائهٔ فنی پروژهٔ ریرا

سرویس مدیریت اشخاص با gRPC

در این پروژه، یک سرویس برای ثبت و مدیریت اطلاعات اشخاص ساخته‌ام. هر شخص یک شناسه، نام، نام خانوادگی، کد ملی و تاریخ تولد دارد. سرویس امکان ثبت، ویرایش، حذف و جست‌وجوی این اطلاعات را فراهم می‌کند.

برای ارتباط با کلاینت از gRPC استفاده کرده‌ام و پیام‌ها و متدهای سرویس را در فایل Protobuf تعریف کرده‌ام. اطلاعات با EF Core در یک دیتابیس SQLite ذخیره می‌شوند.

ایدهٔ اصلی طراحی

منطق اعتبارسنجی شخص را در Domain قرار داده‌ام. اجرای هر عملیات در Application انجام می‌شود. کد مربوط به دیتابیس در Infrastructure است و Endpoint ارتباط با کلاینت را مدیریت می‌کند. این جداسازی کمک می‌کند هر بخش مسئولیت مشخصی داشته باشد و بتوانم آن را جداگانه تست کنم.

برای خواندن و تغییر اطلاعات، سرویس‌ها و کلاس‌های Repository ی جدا دارم. این بخش را با CQRS ساده و بدون MediatR پیاده‌سازی کرده‌ام. برای نتیجهٔ عملیات هم از Result استفاده کرده‌ام تا موفقیت و خطاهای قابل انتظار به‌صورت مشخص برگردند.

موضوعات ارائه

در صفحات بعد، مسیر یک درخواست، ساختار لایه‌ها و دلیل انتخاب هر Pattern را توضیح می‌دهم. سپس سراغ دیتابیس، مدیریت خطا و تست‌ها می‌روم. راهنمای اجرا از صفحهٔ ۱۳ شروع می‌شود و نمونه‌های فراخوانی سرویس در صفحات ۱۵ و ۱۶ قرار دارند.

```
.NET 10 / ASP.NET Core gRPC
Protocol Buffers / EF Core / SQLite
xUnit / Moq / WebApplicationFactory
```

مسیر یک درخواست در برنامه

ثبت شخص

کلاینت با متد CreatePerson اطلاعات شخص را می‌فرستد. در Endpoint ، دادهٔ ورودی به DTO تبدیل می‌شود و PersonCommandService اجرای عملیات را ادامه می‌دهد.

سرویس با متد Person.Create یک Person می‌سازد. این متد نام، کد ملی و تاریخ تولد را بررسی می‌کند. اگر ورودی نامعتبر باشد، عملیات همین‌جا با Result ناموفق تمام می‌شود.

اگر اطلاعات معتبر باشد، سرویس تکراری نبودن کد ملی را بررسی می‌کند. سپس Repository شخص را به DbContext اضافه می‌کند و UnitOfWork تغییرات را ذخیره می‌کند. در پایان، اطلاعات ثبت‌شده به DTO و سپس به پاسخ gRPC تبدیل می‌شود.

```
CreatePersonRequest
  -> PersonGrpcService
  -> PersonCommandService
  -> Person.Create
  -> PersonRepository.AddAsync
  -> UnitOfWork.SaveChangesAsync
  -> PersonResponseDto
  -> PersonResponse
```

خواندن اطلاعات

در مسیر Query ، سرویس فقط دادهٔ موردنیاز پاسخ را می‌گیرد. ReadRepository مستقیماً DTO برمی‌گرداند؛ بنابراین برای نمایش اطلاعات، Person ساخته یا Track نمی‌شود.

اگر شخص پیدا نشود، Application نتیجهٔ NotFound می‌سازد. اگر فهرست اشخاص خالی باشد، عملیات موفق است و فهرست خالی برمی‌گردد.

مسئولیت هر مرحله

Endpoint با قرارداد gRPC کار می‌کند، Application ترتیب اجرای عملیات را مشخص می‌کند، Domain اعتبار اطلاعات را بررسی می‌کند و Infrastructure داده را می‌خواند یا ذخیره می‌کند.

ساختار لایه‌ها

Domain

کلاس Person ، کلاس‌های پایه و کلاس‌های Value Object ی کد ملی و تاریخ تولد در این لایه هستند. قوانین اصلی برنامه اینجا قرار دارند. Domain به SQLite ، EF Core یا gRPC وابسته نیست.

Application

این لایه شامل سرویس‌های Command و Query ، کلاس‌های DTO و Interface های موردنیاز آن‌هاست. مثلاً PersonCommandService مشخص می‌کند برای ویرایش شخص، چه داده‌ای خوانده شود، چه بررسی‌هایی انجام شود و چه زمانی تغییرات ذخیره شوند.

Infrastructure

پیاده‌سازی کلاس‌های UnitOfWork ، DbContext ، Repository ، تنظیمات EF و فایل‌های Migration را در این لایه قرار داده‌ام. Application فقط Interface ها را می‌شناسد؛ جزئیات کار با SQLite اینجا باقی می‌مانند.

Endpoint

این پروژه همان برنامهٔ ASP.NET Core است که اجرا می‌شود. فایل proto ، سرویس gRPC ، کد Mapping و Interceptor در آن قرار دارند. در Program هم سرویس‌ها را در DI Container ثبت کرده‌ام.

```
Endpoint ------> Application ------> Domain
    |                                  ^
    +----------> Infrastructure -------+
                       |
                       +----> Application contracts
```

دلیل این ساختار

با این روش، تغییر قرارداد API یا تنظیمات دیتابیس، لزوماً قواعد Person را تغییر نمی‌دهد. تست‌های Domain هم بدون اجرای Host یا اتصال به دیتابیس اجرا می‌شوند. این پروژه یک برنامهٔ واحد است و لایه‌ها به‌صورت سرویس‌های جدا اجرا نمی‌شوند.

طراحی Person

Entity و شناسه

برای Person یک شناسهٔ Guid در نظر گرفته‌ام. نام یا تاریخ تولد شخص ممکن است تغییر کند، اما شناسهٔ او ثابت می‌ماند. کلاس پایهٔ Entity هم برابری را بر اساس نوع کلاس و شناسه بررسی می‌کند.

Aggregate Root

کلاس Person از AggregateRoot ارث می‌برد. در این مدل، Person واحد مستقلی است که تغییراتش باید از متدهای خودش عبور کند. این موضوع کمک می‌کند قواعد شخص فقط در یک نقطه کنترل شوند.

کلاس پایه امکان نگهداری Domain Event را هم دارد، اما در عملیات فعلی رویدادی تولید یا منتشر نمی‌شود. اجرای CRUD به Message Broker یا Event Dispatcher وابسته نیست.

Factory Method

سازندهٔ Person را private گذاشته‌ام. برای ساخت شخص باید متد Create فراخوانی شود. این متد ابتدا ورودی‌ها را بررسی می‌کند و فقط در صورت معتبر بودن، یک Person می‌سازد. در غیر این صورت، Result ناموفق برمی‌گرداند.

ویرایش کنترل‌شده

برای تغییر اطلاعات از متد Update استفاده می‌کنم. این متد قبل از تغییر هر فیلد، همهٔ ورودی‌های لازم را بررسی می‌کند. مثلاً اگر تاریخ تولد نامعتبر باشد، نام جدید هم روی Person اعمال نمی‌شود.

Setter هر ویژگی private است؛ بنابراین کد بیرون از Person نمی‌تواند با مقداردهی مستقیم، این بررسی‌ها را دور بزند.

قواعد نام

نام و نام خانوادگی پس از Trim نباید خالی باشند. حداکثر طول هرکدام ۱۰۰ کاراکتر است. محدودیتی برای زبان نام تعریف نکرده‌ام.

طراحی Value Object

دلیل استفاده از Value Object

کد ملی و تاریخ تولد هرکدام قواعد مشخصی دارند. به‌جای تکرار این قواعد در سرویس‌ها، برای آن‌ها Value Object ساخته‌ام. مقدار این کلاس‌ها بعد از ساخته شدن تغییر نمی‌کند و برابری آن‌ها به مقدارشان بستگی دارد، نه به شناسه.

NationalCode

متد Create فاصله‌های ابتدا و انتهای ورودی را حذف می‌کند و ارقام فارسی و عربی را به ارقام انگلیسی تبدیل می‌کند. بعد طول، نوع کاراکترها و Checksum را بررسی می‌کند. کد باید دقیقاً ۱۰ رقم باشد و همهٔ رقم‌های آن نباید یکسان باشند.

```
sum = d[0]*10 + d[1]*9 + ... + d[8]*2
r = sum % 11
expected = r < 2 ? r : 11 - r
valid = d[9] == expected
```

کد ملی را به شکل string نگه داشته‌ام تا صفرهای ابتدای آن حذف نشوند. این بررسی فقط ساختار کد را تأیید می‌کند؛ برای تأیید هویت واقعی شخص باید از سرویس استعلام جداگانه استفاده شود.

BirthDate

برای تاریخ تولد از DateOnly استفاده کرده‌ام، چون ساعت جزئی از تاریخ تولد نیست. متد Create مقدار پیش‌فرض و تاریخ آینده را رد می‌کند. تاریخ امروز را Application با زمان تهران محاسبه می‌کند و به Domain می‌دهد.

برابری و دسترسی به مقدار

در کلاس پایهٔ ValueObject ، برابری از روی مقدار محاسبه می‌شود. متدهای Equals و GetHashCode و عملگرهای == و != با همین منطق کار می‌کنند. برای گرفتن مقدار ساده نیز تبدیل صریح به string یا DateOnly وجود دارد.

سازنده‌های این دو کلاس private هستند؛ ساخت نمونهٔ جدید فقط از مسیر Create انجام می‌شود.

Result Pattern

دلیل انتخاب

نام نامعتبر، کد ملی تکراری و پیدا نشدن شخص، اتفاق‌های قابل انتظار در این سرویس هستند. برای این وضعیت‌ها از Result استفاده کرده‌ام تا نتیجهٔ عملیات مشخص باشد و لازم نباشد مسیر عادی برنامه با Exception کنترل شود.

```
Result
  IsSuccess
  Message
  ErrorType

Result<T>
  Value
```

برای عملیاتی مثل حذف، Result ساده کافی است. اگر عملیات خروجی داشته باشد، از Result<T> استفاده می‌کنم؛ مثل ساخت Value Object یا دریافت اطلاعات شخص.

انواع خطا

نوع Validation برای ورودی نامعتبر است. NotFound یعنی شخص پیدا نشده است. Conflict برای تعارض و ServerError برای خطای عمومی ذخیره استفاده می‌شود. Application نتیجه را با همین نوع‌ها برمی‌گرداند و Endpoint آن را به Status مناسب gRPC تبدیل می‌کند.

مرز Result و Exception

از Result برای همهٔ خطاها استفاده نکرده‌ام. خطای غیرمنتظره، استفادهٔ نادرست از قرارداد کد و لغو درخواست همچنان می‌توانند Exception باشند. Interceptor خطاهای مدیریت‌نشده را در مرز gRPC ثبت و به پاسخ عمومی تبدیل می‌کند.

رفتار Repository

اگر رکوردی پیدا نشود، Repository مقدار null برمی‌گرداند و Application تصمیم می‌گیرد آن را به NotFound تبدیل کند. اما خطای SQL به null یا فهرست خالی تبدیل نمی‌شود؛ نبود داده با خرابی دیتابیس دو وضعیت متفاوت است.

کلاس Result در Domain/Common قرار دارد تا Factory های Domain و سرویس‌های Application بتوانند از آن استفاده کنند. این کلاس به gRPC یا EF وابسته نیست.

CQRS بدون MediatR

جداسازی Command و Query

برای تغییر اطلاعات از PersonCommandService و برای خواندن از PersonQueryService استفاده کرده‌ام. Command با Person و قواعد آن کار می‌کند. Query فقط دادهٔ موردنیاز پاسخ را می‌گیرد.

```
Command -> Person -> Write Repository -> UnitOfWork
Query   -> Read Repository -> DTO
```

خروجی Query

خروجی ReadRepository از نوع PersonResponseDto است. برای نمایش اطلاعات، نیازی به متدهای Person یا Tracking آن ندارم. این انتخاب همچنین مانع می‌شود ساختار داخلی Domain مستقیماً وارد پاسخ API شود.

چرا سرویس عادی؟

در این پروژه تعداد عملیات کم است و Interface سرویس‌ها برای فراخوانی آن‌ها کافی است. CQRS به MediatR وابسته نیست؛ اصل آن جداسازی مسئولیت خواندن و نوشتن است. با فراخوانی مستقیم، مسیر اجرا ساده و قابل دنبال کردن باقی می‌ماند.

یک دیتابیس مشترک

هر دو مسیر از یک دیتابیس SQLite استفاده می‌کنند. جداسازی در سطح سرویس، Repository و مدل خروجی انجام شده است. برای این پروژه به دیتابیس جدا یا هماهنگ کردن دو نسخه از داده نیاز نداشته‌ام.

DTO و Mapping

برای ورودی و خروجی سرویس‌ها از DTO استفاده کرده‌ام. DTO فقط داده را منتقل می‌کند و رفتار Person را ندارد. Mapping را دستی نوشته‌ام، چون تعداد فیلدها کم است و می‌خواهم تبدیل تاریخ و کلاس‌های Value Object در کد واضح باشد.

این پیاده‌سازی، CQRS ساده است. Event Sourcing یا Read Replica در آن وجود ندارد.

Repository و Dependency Injection

Repository مربوط به Command

قرارداد IPersonRepository در Domain قرار دارد. این Interface عملیات موردنیاز Person را تعریف می‌کند: خواندن با شناسه یا کد ملی، افزودن، ویرایش و حذف. نوع‌های EF و IQueryable از این قرارداد بیرون داده نمی‌شوند.

ReadRepository

قرارداد IPersonReadRepository در Application است، چون خروجی آن DTO متعلق به همین لایه است. با این تصمیم، Domain برای تعریف Repository خود به مدل پاسخ Application وابسته نمی‌شود.

Unit of Work

متدهای Update ، Add و Remove تغییرات را آماده می‌کنند، اما آن‌ها را بلافاصله ذخیره نمی‌کنند. Application در پایان عملیات، SaveChangesAsync را فراخوانی می‌کند. بنابراین زمان ذخیره مشخص است و مدیریت خطای ذخیره هم در یک محل انجام می‌شود.

Dependency Inversion

سرویس Application به Interface وابسته است، نه به کلاس Repository مربوط به .SQLite پیاده‌سازی Interface در Infrastructure قرار دارد و در Program به آن وصل می‌شود. این کار امکان تست سرویس با Mock را فراهم می‌کند.

طول عمر سرویس‌ها

برای DbContext ، کلاس‌های UnitOfWork ، Repository و سرویس‌های Application از Scoped استفاده کرده‌ام. اجزای یک درخواست از DbContext مشترک استفاده می‌کنند. این Context نباید بین چند عملیات موازی به اشتراک گذاشته شود.

چرا Repository عمومی ندارم؟

قراردادها را متناسب با نیاز Person نوشته‌ام. هدف، ایجاد یک مرز مشخص برای دسترسی داده است؛ نه تکرار همهٔ قابلیت‌های EF در یک .Generic Repository تغییر دیتابیس همچنان به بررسی Query ها، فایل‌های Migration و تست‌ها نیاز دارد.

EF Core و طراحی دیتابیس

چرا SQLite ؟

برای اجرای این پروژه به دیتابیس سبک و قابل راه‌اندازی نیاز داشتم. SQLite بدون نصب یک سرویس دیتابیس جدا کار می‌کند و داده را در فایل نگه می‌دارد. در تست‌ها هم می‌توانم از همان Provider استفاده کنم.

Entity Configuration

تنظیمات Person را در PersonConfiguration قرار داده‌ام. نام جدول، کلید، محدودیت‌ها و تبدیل کلاس‌های Value Object اینجا تعریف شده‌اند. این تنظیمات وارد Domain نشده‌اند.

```
Persons
  Id           TEXT  PRIMARY KEY
  FirstName    TEXT  NOT NULL
  LastName     TEXT  NOT NULL
  NationalCode TEXT  NOT NULL, UNIQUE
  BirthDate    TEXT  NOT NULL
```

Value Converter

برای ذخیرهٔ NationalCode ، مقدار string آن را می‌گیرم. BirthDate هم به DateOnly تبدیل می‌شود. هنگام خواندن Person ، این مقدارها دوباره به Value Object تبدیل می‌شوند.

شناسه در Person ساخته می‌شود؛ به همین دلیل در EF از ValueGeneratedNever استفاده کرده‌ام.
Domain Event ها هم در جدول ذخیره نمی‌شوند.

Constraint و Index

برای طول نام‌ها و ساختار کد ملی، Check Constraint دارم. برای کد ملی نیز Unique Index تعریف کرده‌ام.
بررسی اولیهٔ سرویس برای پاسخ مناسب است، اما تضمین نهایی جلوگیری از کد تکراری را دیتابیس انجام می‌دهد.

Tracking در دو مسیر

Repository مربوط به Command یک Person را Track می‌کند تا تغییراتش در همان Context ذخیره شود.
ReadRepository مستقیماً DTO می‌سازد و Person را Track نمی‌کند. ورودی جست‌وجو هم با الحاق رشته به SQL وارد نمی‌شود.

Transaction و مدیریت شکست ذخیره

ذخیره در یک مرحله

در هر Command ، تغییرات آماده می‌شوند و سپس یک SaveChangesAsync اجرا می‌شود. در رفتار پیش‌فرض EF و Provider تراکنشی، تغییرات همان فراخوانی با هم ذخیره می‌شوند. اگر ذخیره شکست بخورد، Transaction همان تغییرات را Rollback می‌کند.

نقش DiscardChanges

Rollback مربوط به دیتابیس است. DiscardChanges برای وضعیت اشیای داخل حافظه استفاده می‌شود. این متد مقدارهای قبلی اشیای تغییرکرده را برمی‌گرداند و ChangeTracker را پاک می‌کند تا تغییرات ناموفق در عملیات بعدی ذخیره نشوند.

مسئولیت Cleanup

اگر قبل از شروع ذخیره، آماده‌سازی تغییرات خطا بدهد، Application درخواست Cleanup می‌کند. پس از شروع SaveChangesAsync ، خود UnitOfWork مسئول Cleanup در مسیر شکست است. برای برگرداندن وضعیت، متد Person.Update دوباره اجرا نمی‌شود.

Unique Constraint و Concurrency

خطای Unique Constraint به Conflict تبدیل می‌شود. برای ویرایش و حذف هم ویژگی‌های Person را به‌عنوان Concurrency Token تنظیم کرده‌ام تا تغییر داده بین زمان خواندن و ذخیره در همان درخواست تشخیص داده شود.

این کنترل مربوط به فاصلهٔ Load تا Save است. درخواست فعلی Update ، نسخهٔ داده‌ای را که کاربر قبلاً دیده است همراه ندارد. جلوگیری از ثبت فرم قدیمی به کنترل Version در قرارداد API نیاز دارد.

Exception و Retry

خطای عمومی ذخیره Log می‌شود و ServerError برمی‌گردد. Cancellation پس از Cleanup ادامه پیدا می‌کند. Retry خودکار و بدون بررسی انجام نمی‌دهم؛ در تعارض ویرایش، داده باید دوباره خوانده شود و عملیات با وضعیت جدید تصمیم‌گیری شود.

قرارداد gRPC و مدیریت خطا

Protobuf

پیام‌ها و متدهای سرویس در persons.proto تعریف شده‌اند. ابزار gRPC از این فایل، کلاس‌های Client و Server را تولید می‌کند. Endpoint درخواست را می‌گیرد و اجرای عملیات را به سرویس Application می‌سپارد.

```
package rira.persons.v1;
CreatePerson / UpdatePerson / DeletePerson
GetPersonById / GetPersonByNationalCode / GetAllPersons
```

Mapping تاریخ و شناسه

شناسهٔ ورودی قبل از اجرای عملیات به Guid تبدیل و بررسی می‌شود. تاریخ تولد با سه فیلد year ، month و day ارسال و به DateOnly تبدیل می‌شود. تاریخ API میلادی است و ساعت ندارد.

در Update باید همهٔ فیلدها ارسال شوند. ارسال نکردن یک فیلد، به معنی حفظ مقدار قبلی آن نیست.

تبدیل Result به Status

```
Validation  -> InvalidArgument
NotFound    -> NotFound
Conflict    -> AlreadyExists
ServerError -> Internal
```

این جدول رفتار فعلی Endpoint را نشان می‌دهد. پیام خطای داخلی دیتابیس به کلاینت ارسال نمی‌شود.

Interceptor

برای مدیریت خطاهای عمومی از GrpcExceptionInterceptor استفاده کرده‌ام. اگر RpcException از قبل ساخته شده باشد، همان را عبور می‌دهد. خطای غیرمنتظره را همراه نام متد و TraceIdentifier ثبت می‌کند و پاسخ عمومی Internal می‌دهد.

CancellationToken از درخواست تا Repository و SaveChangesAsync منتقل می‌شود. در مسیر لغو درخواست، پاسخ Cancelled یا DeadlineExceeded برمی‌گردد. Reflection فعال نشده؛ بنابراین ابزارهایی مثل grpcurl باید فایل proto را دریافت کنند.

تست‌های پروژه

Domain Tests

این تست‌ها بدون دیتابیس اجرا می‌شوند. اعتبار نام‌ها، Checksum کد ملی، تبدیل ارقام، تاریخ تولد، برابری کلاس‌های Value Object و رفتار Person هنگام ورودی نامعتبر را بررسی می‌کنند.

Application Tests

برای تست سرویس‌ها از Moq استفاده کرده‌ام. در این بخش نتیجهٔ موفق، NotFound ، کد تکراری، خروجی DTO و انتقال CancellationToken بررسی می‌شود. همچنین تست می‌کنم که Save و Cleanup در مسیر درست اجرا شوند.

Infrastructure Tests

در این تست‌ها از SQLite واقعی استفاده می‌کنم. Constraint ، Migration ها، Value Converter ها، Tracking و Rollback با Provider اصلی پروژه بررسی می‌شوند.

برای رقابت دو درج، تستی با دو Connection مستقل وجود دارد. هر دو درخواست از بررسی اولیه عبور می‌کنند، اما به‌دلیل Unique Index فقط یکی باید ذخیره شود.

Endpoint Tests

با WebApplicationFactory برنامه را در TestServer اجرا می‌کنم. Client تولیدشده از proto به آن درخواست می‌فرستد. به این ترتیب مسیر Application ، Mapping ، gRPC و دیتابیس با هم تست می‌شوند.

نتیجهٔ اجرای تست‌ها

```
dotnet test Rira.slnx -c Release --nologo --verbosity minimal
Passed: 330 | Failed: 0 | Skipped: 0
```

مسیر CRUD روی Kestrel واقعی و دیتابیس فایل‌محور نیز اجرا شده است. این تست‌ها رفتار عملیات تعریف‌شده را بررسی می‌کنند؛ تست بار، عملکرد و شرایط استقرار، بررسی‌های جداگانه هستند.

اجرای پروژه

دستورها باید در پوشهٔ دارای Rira.slnx اجرا شوند. دستور اول برای حالتی است که پوشهٔ Rira داخل پوشهٔ فعلی قرار دارد. اگر از قبل کنار فایل Solution هستید، Set-Location را اجرا نکنید. برای اجرا به SDK دات‌نت ۱۰ نیاز دارید.

```
Set-Location .\Rira
dotnet --version
dotnet restore Rira.slnx
dotnet tool restore
dotnet build Rira.slnx -c Release
dotnet test Rira.slnx -c Release --no-build
```

۱. تعیین دیتابیس مشترک

```
$dbPath = Join-Path (Get-Location).Path "rira.db"
$connection = "Data Source=$dbPath"
$env:ConnectionStrings__Rira = $connection
```

۲. ساخت Schema با Migration موجود

```
dotnet ef database update `
  --project Rira.Infrastructure `
  --startup-project Rira.Endpoint `
  --configuration Release --no-build `
  --connection "$connection"
```

۳. اجرای Host

```
dotnet run --project Rira.Endpoint `
  --configuration Release --no-build --launch-profile http
```

سرویس با آدرس localhost:5120 و HTTP/2 اجرا می‌شود. این ترمینال را باز نگه دارید و درخواست‌ها را از ترمینال دوم بفرستید. برای توقف برنامه Ctrl+C بزنید. متغیر ConnectionStrings__Rira فقط در همین ترمینال و برنامه‌های اجراشده از آن در دسترس است.

مسیر دیتابیس از پوشهٔ جاری محاسبه می‌شود. همان Connection String را به ابزار EF و Host می‌دهیم تا هر دو از یک فایل استفاده کنند. این راهنما برای اجرای HTTP روی سیستم محلی است.

مدیریت Migration و Publish

بررسی وضعیت Schema

```
dotnet ef migrations list `
  --project Rira.Infrastructure `
  --startup-project Rira.Endpoint `
  --connection "$connection"

dotnet ef migrations has-pending-model-changes `
  --project Rira.Infrastructure `
  --startup-project Rira.Endpoint
```

پروژه از قبل Migration اولیه را دارد. برای ساخت دیتابیس جدید، database update کافی است و لازم نیست Migration دیگری بسازید. دستور has-pending-model-changes اختلاف مدل فعلی با آخرین Migration را بررسی می‌کند.

فقط پس از تغییر عمدی مدل

```
dotnet ef migrations add DescribeModelChange `
  --project Rira.Infrastructure `
  --startup-project Rira.Endpoint `
  --output-dir Persistence/Migrations
```

دستور migrations add فقط زمانی لازم است که مدل یا تنظیمات EF را تغییر داده‌اید. فایل تولیدشده را بررسی کنید و بعد database update را اجرا کنید. این پروژه از Migration استفاده می‌کند؛ مسیر EnsureCreated را با آن ترکیب نکنید.

Factory و تنظیمات زمان طراحی

کلاس RiraDbContextFactory برای ابزار EF یک DbContext می‌سازد. این کلاس تنظیمات Host را نمی‌خواند. به همین دلیل، برای database update و migrations list مقدار --connection را صریح می‌فرستیم.

Publish برنامه

```
dotnet publish Rira.Endpoint -c Release -o ./artifacts/app
# Keep ConnectionStrings__Rira set to the absolute path.
Push-Location ./artifacts/app
dotnet Rira.Endpoint.dll --urls http://localhost:5120
# After stopping the host with Ctrl+C:
Pop-Location
```

برای اجرای نسخهٔ Publish شده، وارد پوشهٔ خروجی شوید تا appsettings همان پوشه خوانده شود. قبل از اجرا، متغیر ConnectionStrings__Rira را تنظیم کنید. فایل launchSettings فقط برای اجرای توسعه است.

فراخوانی سرویس؛ Create و Query

برای ارسال درخواست دستی می‌توانید از grpcurl استفاده کنید. این ابزار جداگانه نصب می‌شود. ترمینال دوم را کنار Rira.slnx باز کنید. اگر از پوشهٔ والد وارد می‌شوید، Set-Location را اجرا کنید. چون Reflection فعال نیست، فایل proto را با آرگومان معرفی می‌کنیم.

آماده‌سازی ترمینال دوم

```
Set-Location .\Rira
$rpcArgs = @(
  "-plaintext",
  "-import-path", "Rira.Endpoint/Protos",
  "-proto", "persons.proto"
)
$service = "rira.persons.v1.PersonService"
```

ایجاد شخص

```
$request = @{
  firstName = "Ali"
  lastName = "Ahmadi"
  nationalCode = "0012345679"
  birthDate = @{ year = 1995; month = 5; day = 21 }
} | ConvertTo-Json -Compress

$raw = $request | grpcurl @rpcArgs -d "@" `
  localhost:5120 "$service/CreatePerson"
if ($LASTEXITCODE -ne 0) { throw "Create failed" }
$created = $raw | ConvertFrom-Json
$personId = $created.id
```

خواندن با شناسه و کد ملی

```
@{ id = $personId } | ConvertTo-Json -Compress |
  grpcurl @rpcArgs -d "@" localhost:5120 `
    "$service/GetPersonById"

@{ nationalCode = "0012345679" } |
  ConvertTo-Json -Compress |
  grpcurl @rpcArgs -d "@" localhost:5120 `
    "$service/GetPersonByNationalCode"
```

شناسهٔ پاسخ Create در متغیر personId قرار می‌گیرد و در درخواست‌های بعدی استفاده می‌شود. اگر همین کد ملی از قبل ثبت شده باشد، Create با AlreadyExists تمام می‌شود؛ در این حالت شناسه را با Query بگیرید. کد ملی نمونه فقط دادهٔ تست است.

فراخوانی سرویس؛ Update و Delete

این دستورها را در همان ترمینال صفحهٔ قبل اجرا کنید تا متغیرهای service ، rpcArgs و personId در دسترس باشند. در درخواست Update همهٔ فیلدها را بفرستید.

ویرایش و دریافت فهرست

```
$update = @{
  id = $personId
  firstName = "Maryam"
  lastName = "Rezaei"
  nationalCode = "1234567891"
  birthDate = @{ year = 2000; month = 2; day = 29 }
} | ConvertTo-Json -Compress

$update | grpcurl @rpcArgs -d "@" localhost:5120 `
  "$service/UpdatePerson"

"{}" | grpcurl @rpcArgs -d "@" localhost:5120 `
  "$service/GetAllPersons"
```

حذف و بررسی نبود رکورد

```
@{ id = $personId } | ConvertTo-Json -Compress |
  grpcurl @rpcArgs -d "@" localhost:5120 `
    "$service/DeletePerson"

@{ id = $personId } | ConvertTo-Json -Compress |
  grpcurl @rpcArgs -d "@" localhost:5120 `
    "$service/GetPersonById"
```

حذف موفق پاسخ Empty دارد. بعد از حذف، Query با همان شناسه باید NotFound برگرداند. اگر هیچ شخصی باقی نمانده باشد، GetAll موفق است و فهرست خالی دارد. نمایش JSON ممکن است فیلد فهرست خالی را حذف کند.

نمونهٔ خطای ورودی

```
@{ id = "not-a-guid" } | ConvertTo-Json -Compress |
  grpcurl @rpcArgs -d "@" localhost:5120 `
    "$service/GetPersonById"
```

برای شناسهٔ نامعتبر، پاسخ InvalidArgument انتظار می‌رود. Status درخواست gRPC را بررسی کنید؛ HTTP Status به‌تنهایی نتیجهٔ عملیات را نشان نمی‌دهد. برای دیدن خطای Duplicate ، قبل از حذف همان درخواست Create را دوباره ارسال کنید.

برنامهٔ نمایش عملی پروژه

شروع ارائه

ابتدا مسئله را توضیح می‌دهم: سرویس باید اطلاعات شخص را با Protobuf دریافت و مدیریت کند. سپس چهار لایه را نشان می‌دهم و مسیر یک درخواست را از Endpoint تا دیتابیس توضیح می‌دهم.

ثبت و خواندن

یک شخص با کد ملی معتبر ثبت می‌کنم. در نام، فاصلهٔ اضافی می‌گذارم تا نتیجهٔ Trim در پاسخ دیده شود. بعد همان شخص را با شناسه و کد ملی می‌خوانم.

نمایش Validation

دوباره همان کد ملی را می‌فرستم تا پاسخ AlreadyExists نمایش داده شود. سپس یک تاریخ یا شناسهٔ نامعتبر می‌فرستم تا InvalidArgument دیده شود. دادهٔ قبلی را دوباره می‌خوانم تا مشخص شود درخواست نامعتبر چیزی را تغییر نداده است.

ویرایش و حذف

اطلاعات شخص را ویرایش می‌کنم و نتیجه را با Query می‌خوانم. سپس او را حذف می‌کنم. Query بعد از حذف باید NotFound برگرداند و GetAll باید فهرست خالی بدهد.

نمایش بخش‌های مهم کد

متد Create در Person ، کلاس‌های Result ، Value Object و تفاوت دو Repository را نشان می‌دهم. بعد توضیح می‌دهم که SaveChangesAsync فقط در UnitOfWork اجرا می‌شود و خطای ذخیره چطور مدیریت می‌شود.

پایان ارائه

تست‌ها را اجرا می‌کنم و چند سناریوی مهم، مثل ورودی نامعتبر، کد ملی تکراری و شکست ذخیره را توضیح می‌دهم. در جمع‌بندی، روی مسئولیت هر لایه و دلیل انتخاب این ساختار تمرکز می‌کنم.

پرسش و پاسخ؛ تصمیم‌های معماری

چرا این لایه‌بندی؟

می‌خواستم قواعد Person به دیتابیس یا gRPC وابسته نباشند. به همین دلیل، اعتبار داده در Domain ، اجرای عملیات در Application و جزئیات بیرونی در Infrastructure و Endpoint قرار گرفته‌اند.

چرا CQRS بدون MediatR ؟

برای جداسازی خواندن و نوشتن، به MediatR نیاز ندارم. در این پروژه Interface سرویس‌ها و کلاس‌های Repository ی جدا کافی‌اند و فراخوانی مستقیم، مسیر اجرا را روشن نگه می‌دارد.

چرا ReadRepository در Application است؟

چون خروجی آن DTO متعلق به Application است. قرار دادن این قرارداد در Domain ، باعث می‌شد Domain به مدل پاسخ Application وابسته شود.

چرا Value Object ؟

کد ملی و تاریخ تولد قواعد خاص دارند. Value Object این قواعد را در یک محل نگه می‌دارد. سازندهٔ private و متد Create هم اجازه نمی‌دهند مصرف‌کننده یک نمونهٔ نامعتبر بسازد.

چرا Aggregate Root ؟

در مدل فعلی، Person مستقل خوانده و تغییر داده می‌شود. تغییرات آن باید از متدهای خودش عبور کند. کلاس پایه امکان نگهداری Domain Event را هم دارد، ولی CRUD فعلی از انتشار رویداد استفاده نمی‌کند.

چرا Mapping دستی؟

تعداد فیلدها کم است و تبدیل‌ها ساده‌اند. Mapping دستی کمک می‌کند تبدیل Guid ، DateOnly و کلاس‌های Value Object در کد مشخص باشد و به تنظیمات یک کتابخانهٔ دیگر وابسته نباشد.

پرسش و پاسخ؛ داده و مدیریت خطا

چرا هم بررسی تکراری بودن و هم Unique Index ؟

بررسی اولیه برای پاسخ سریع و مناسب است. اما دو درخواست ممکن است هم‌زمان از آن عبور کنند. Unique Index جلوی ثبت دو کد یکسان را در خود دیتابیس می‌گیرد.

چرا Result و Exception را با هم دارید؟

Result برای وضعیت‌های قابل انتظار مثل Validation و NotFound است. Exception برای خطای غیرمنتظره و Cancellation باقی می‌ماند. در Endpoint هر دو مسیر به پاسخ مناسب gRPC می‌رسند.

Repository چه چیزی به EF اضافه می‌کند؟

یک قرارداد کوچک و متناسب با Person به Application می‌دهد و تست سرویس با Mock را ممکن می‌کند. هدفم تکرار تمام API های EF نبوده است.

DiscardChanges همان Rollback است؟

خیر. Rollback مربوط به Transaction دیتابیس است. DiscardChanges وضعیت اشیایی را که EF آن‌ها را Track می‌کند، پاک می‌کند تا تغییرات ناموفق دوباره ذخیره نشوند. این متد، Commit قبلی یا پیام ارسال‌شده به یک سرویس دیگر را برنمی‌گرداند.

Concurrency چه چیزی را کنترل می‌کند؟

تغییر داده بین Load و Save در یک درخواست را بررسی می‌کند. نسخهٔ داده‌ای که کاربر قبلاً خوانده در درخواست فعلی نیست؛ کنترل فرم قدیمی به Version در قرارداد API نیاز دارد.

چرا تست با SQLite واقعی؟

Mock رفتار Migration ، Constraint یا Transaction واقعی را نشان نمی‌دهد. برای این موارد از SQLite استفاده کرده‌ام. برای تست منطق سرویس، Mock مناسب است و برای مسیر کامل درخواست، Client واقعی gRPC را به TestServer متصل کرده‌ام.

منابع فنی

منابع رسمی زیر، مفاهیم و ابزارهای استفاده‌شده در پروژه را توضیح می‌دهند. برای جزئیات Transaction ، Migration ، Concurrency و قرارداد gRPC می‌توان به آن‌ها مراجعه کرد.

[1] EF Core: Handling Concurrency Conflicts
https://learn.microsoft.com/en-us/ef/core/saving/concurrency

[2] EF Core: Design-time DbContext Creation
https://learn.microsoft.com/en-us/ef/core/cli/dbcontext-creation

[3] gRPC: Status Codes
https://grpc.io/docs/guides/status-codes/

[4] EF Core: Transactions
https://learn.microsoft.com/en-us/ef/core/saving/transactions

[5] Microsoft: Simplified CQRS and DDD
https://learn.microsoft.com/en-us/dotnet/architecture/
microservices/microservice-ddd-cqrs-patterns/
apply-simplified-microservice-cqrs-ddd-patterns

[6] grpcurl: Official repository and usage
https://github.com/fullstorydev/grpcurl

[7] Microsoft.Data.Sqlite: Async limitations
https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/async

[8] ASP.NET Core gRPC
https://learn.microsoft.com/en-us/aspnet/core/grpc/aspnetcore

[9] Protocol Buffers: proto3 Language Guide
https://protobuf.dev/programming-guides/proto3/

راهنمای اجرای این سند با فایل Solution ، تنظیمات Endpoint ، قرارداد persons.proto و Migration موجود در پروژه نوشته شده است.